



微机原理及接口技术 8086 最小模式系统报告

学 生 学 号	22375413
学 生 姓 名	夏 思 博
指 导 教 师	李 妮

2024 年 12 月

1 任务内容

此项目要求在 Proteus 软件中搭建电路原理图。课件中附有所需的元器件列表，可以在 Proteus 中查找并添加到自己的元器件列表中；同时附有汇编程序，需要在 Proteus 项目中的源程序中（main.asm）编译、仿真运行。查找相关资料，添加虚拟示波器或者逻辑分析仪，观察和记录指令执行时的地址总线、数据总线（注意位数）、ALE、~BHE、~RD、~WR、M/~IO 等信号的波形。

内容扩展：增加 8255A、8253、8259 等芯片，设计并开发芯片综合应用，并编写相应的汇编程序代码。例如但不限于电子时钟、交通灯控制系统、波形发生器，具体要求可见课件。

2 电路原理图

2.1 整体电路图

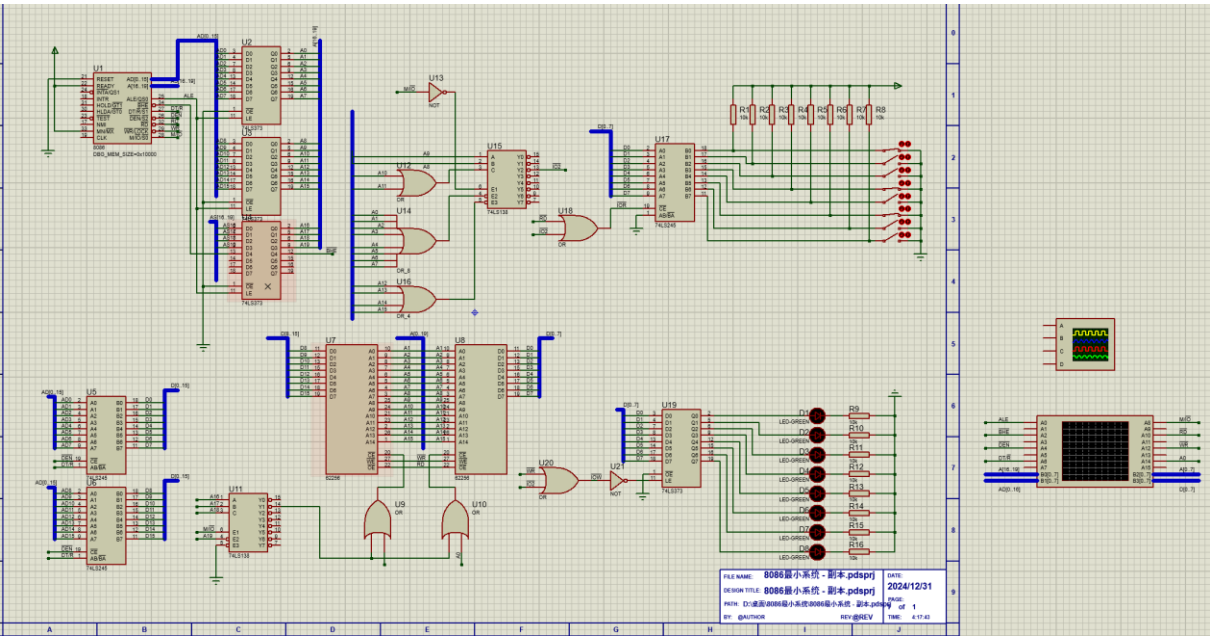


图 1 8086 最小系统 1-6 部分电路图

2.2 交通灯控制电路图

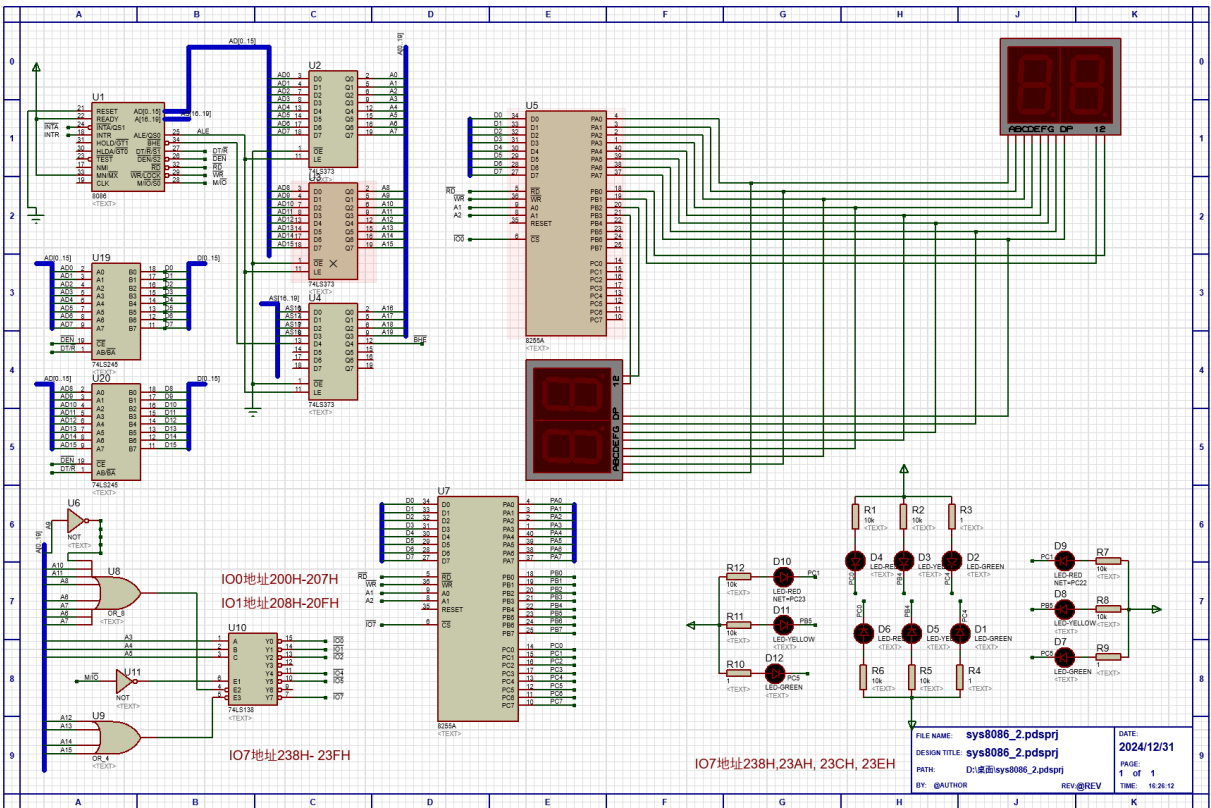


图 2 交通灯控制电路图

3 实现原理

3.1 基本功能实现原理

8086 最小系统基本工作原理：

- (1) 执行读存储器操作。根据 BIU 中的现行段地址寄存器 CS 和指令指针寄存器 IP 的值,在地址加法器中形成 20 位物理地址,通过地址总线送到存储器,指向指定的内存单元,再通过总线控制逻辑电路发出存储器读命令,从给定的地址单元中取出指令,送到先进先出的指令队列中等待执行。
- (2) 执行单元 EU 从指令队列中取走指令,经 EU 控制器进行译码分析后,向各部件发控制命令,以完成执行指令的操作。此时 EU 不需要使用外部总线, BIU 可将多达 6 字节的后继指令送到指令队列,将指令队列填满。
- (3)当指令队列已满,执行单元 EU 在执行指令,未向总线接口单元 BIU 申请读/写存储器或 I/O 操作时,BIU 就处于空闲状态。

(4) 在指令的执行过程中,若需要对存储器或 I/O 端口进行存取数据的操作,EU 就要求 BIU 去完成相应的总线周期。例如,EU 执行到从存储单元中读出一个数据的指令时,它就将操作数的偏移地址,通过内部 16 位数据总线送到 BIU,与 BIU 中的段地址一起,通过地址加法器 Z 形成存储单元的物理地址,再从指定的存储器中取出数据送到 EU,EU 控制器根据指令要求,向内部发控制命令,完成存储器读总线周期。

(5) 在指令的执行过程中,如遇到跳转指令 JMP 或过程调用指令 CALL,须将指令队列中的内容作废,按指令中指定的新的转移地址取指令。

(6) 算术逻辑部件 ALU 完成算术运算、逻辑运算或移位等操作。参加运算的操作数可以从外部的存储器或 I/O 端口获得,也可以从 EU 内部的寄存器等获取。运算结果由 EU 的内部总线送到 EU 或 BIU 的寄存器中,也可由 BIU 写入存储器或 I/O 端口,本次操作的状态(如是否有进位等)反映在标志寄存器 FLAGS 中。

3.2 扩展功能交通信号灯实现原理

使用两个 8255A 接口芯片,分别控制数码管显示数字、红绿黄信号灯交替闪烁。

其中,信号灯引脚接低电平有效。

通过多次循环使得交替闪烁的信号灯可以被清晰的看见,循环的次数就是数字闪动一次所需要的时间,经过尝试,大概循环 30 次左右为 1s。当信号灯绿灯读秒为 0 时候(大循环 17 次),信号灯的颜色跳转,黄灯亮。

数码管上显示数字的程序再显示 3 次,黄灯读秒为 0,所有的红灯全亮,绿灯全灭,进行消隐。然后改变南北向和东西向的红色和绿色。如此循环。

4 系统测试及结果

4.1 基本功能测试

对 1、2、3 部分进行仿真后,使用逻辑分析仪来观察和记录指令执行时的地址总线、数据总线、ALE、BHE、RD、WR、M/IO、DEN、DT/R 等信号的波形,最后得到的结果如下图所示:

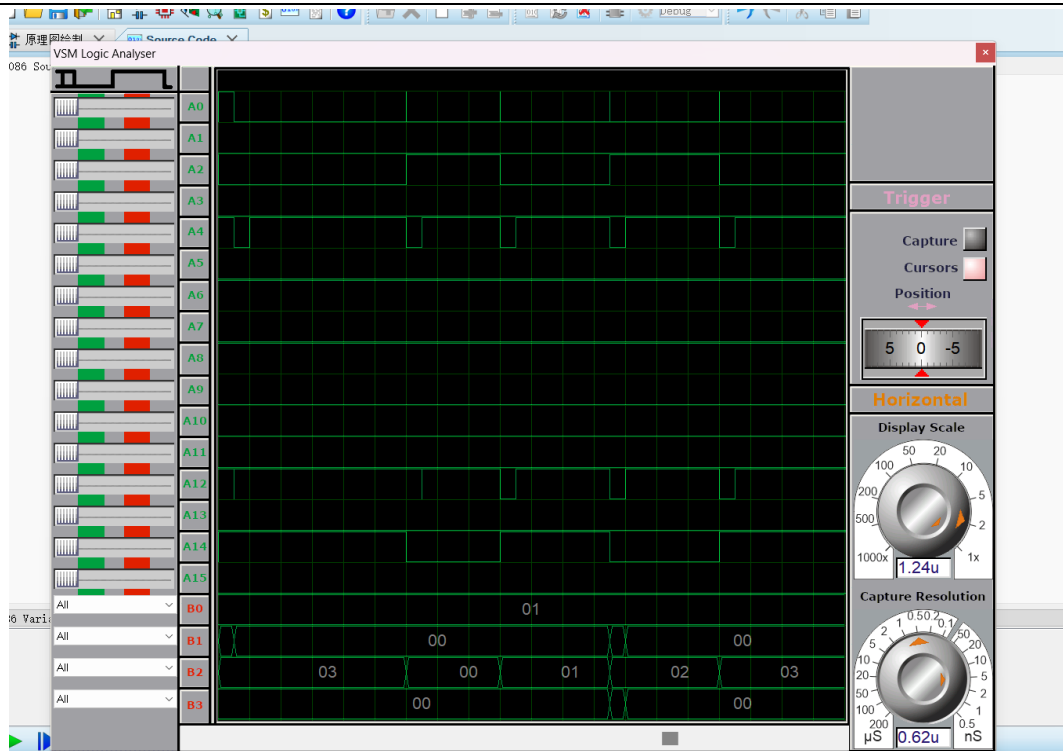


图 3 逻辑分析仪测试图

其中 A0 接的是 ALE, A2 是 $\overline{BHE}$, A4 是 $\overline{DEN}$, A6 是 $DT/\overline{R}$, A8 是 $M/\overline{IO}$, A10 是 $\overline{RD}$, A12 是 $\overline{WR}$, A14 是 A0, B0 是 A[16..19], B1 是 AD[0..16], B2 是 A[0..7], B3 是 D[0..7]。

执行下述指令, 在对 DS 赋 1000H 和对 SI 赋 0 后, 重复执行 SIM 里面的指令。

```

CODE SEGMENT
    ASSUME CS:CODE
START:
    MOV AX,1000H
    MOV DS,AX
    MOV SI, 0

SIM:   MOV BYTE PTR [SI], 01H
        MOV BYTE PTR [SI+1], 02H
        MOV BYTE PTR [SI+2], 03H
        MOV BYTE PTR [SI+3], 04H
        JMP SIM

CODE ENDS
    END START

```

在执行 `MOV BYTE PTR [SI], 01H` 时 $\overline{BHE}=1.A0=0$ ，往偶存储器里写如数据 01H，此时的 $\overline{WR}=0$ ， $DT/\overline{R}=1$

在执行 `MOV BYTE PTR [SI+1], 02H` 时 $\overline{BHE}=0.A0=1$ ，往偶存储器里写如数据 02H，此时的 $\overline{WR}=0$ ， $DT/\overline{R}=1$

在执行 `MOV BYTE PTR [SI+2], 03H` 时 $\overline{BHE}=1.A0=0$ ，往偶存储器里写如数据 03H，此时的 $\overline{WR}=0$ ， $DT/\overline{R}=1$

在执行 `MOV BYTE PTR [SI+3], 04H` 时 $\overline{BHE}=0.A0=1$ ，往偶存储器里写如数据 04H，此时的 $\overline{WR}=0$ ， $DT/\overline{R}=1$

其中偶存储器的物理地址分别为 10000H、10002H

奇存储器的物理地址分别为 10001H、10003H

对 4、5、6 部分进行仿真后, 执行以下程序:

```
CODE SEGMENT
    ASSUME CS:CODE
START:
    MOV DX, 200H
    IN AL, DX
    NOT AL
    OUT DX, AL
    JMP START

CODE ENDS
END START
```

闭合开关，对应的发光二极管亮，如下图所示：

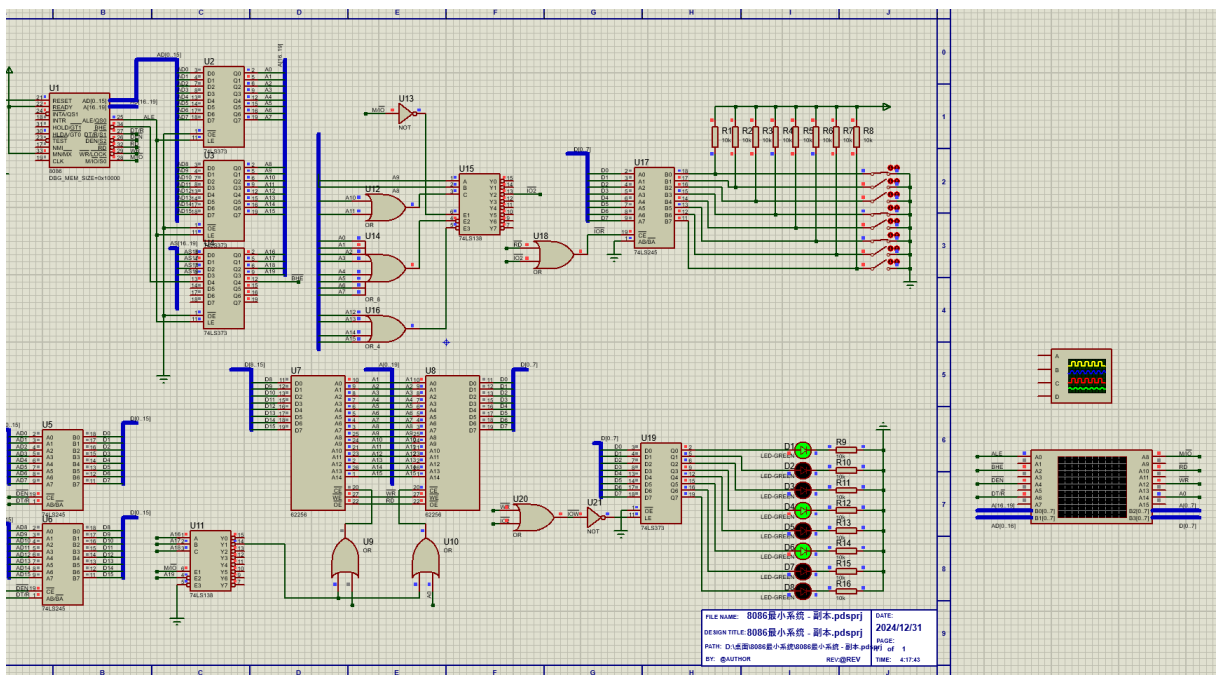


图 4 4-6 部分控制灯的亮灭

在十字路口的东西向、南北向分别装有红、绿、黄指示灯。程序实现如下功能：

8

5 总结与思考

5.1 问题和困难

在进行仿真的时候总是出现 8086 不能进行仿真，要新版的 i8086.DLL 同时复制到 ProgramFiles 和 ProgramData 的 MODELS 目录下，覆盖原来文件。而且 8.12 的 Proteus 有闪退的问题，在 CSDN 上成功解决

在对线进行标号时无法打出低电平有效的符号，最后是用 \$\$ 符号来进行标注。

对线进行标注时发现同一总线的不同电位进行标注时需要重复操作，可以点击键盘 A 键后输入 NET=AD#, 就可以来一次性标注多个线。

5.2 收获和体会

通过这一项目，我巩固了在课堂上学到的 8086 微处理器的工作原理、指令系统、总线控制等理论知识，还学会了如何在 Proteus 软件中将这些理论知识转化为实际的电路设计和程序编写。动手搭建和光看有很大区别，出现了各种各样的 bug，我觉得微机原理是一门硬件与软件，实践与理论结合的课。

5.3 思考题

思考：奇偶存储体的起始物理地址是多少？理解地址生成电路的作用。

答：偶存储器的起始物理地址为 10000H，奇存储器的起始物理地址为 10001H

地址生成电路的主要作用是生成存储器访问的物理地址。在计算机系

统中，地址生成电路通过将基址寄存器、变址寄存器和位移字段的内容相加，得到主存的有效地址，从而帮助实现大容量存储器的寻址和动态程序浮动。